Maintainer Prioritization in npm Supply Chains: Coverage, Exposure, and Graph Representation

Shahriar Hasan*, Ashikul Islam†
*Department of ECE, †Department of CSE
Hajee Mohammad Danesh Science and Technology University

***Abstract*—Limited defensive budgets motivate prioritizing npm maintainers by downstream reach. However, reaching many repositories and reducing exposure to possible account compromises are different objectives. We study this distinction in our collected graph of 25,228 package nodes and 12,217 maintainer usernames. We compare five deterministic selection methods and 200 random orders across six budgets, using package and workflow-associated repository endpoints. At 100 identities, marginal coverage greedy reaches 5,538 of 7,171 observed workflow repositories (77.2%), versus 4,407 for exact maintainer-union ranking. Under a stipulated uniform single-account compromise model with perfect source hardening, their exposure reductions are 11.48% and 33.79%, respectively. Only 800 repositories covered by greedy satisfy exclusion of every observed source identity. Preserving version continuity reduces identity–repository exposure pairs by 42.44%, although aggregate greedy coverage remains similar. These results extend established maintainer-reach analyses with an explicit comparison of objectives and observation units. They support objective-specific prioritization and qualified structural measurements; they do not estimate incident probabilities or demonstrated control effectiveness.**

***Index Terms*—software supply chain, npm, maintainer prioritization, dependency graphs, workflow exposure, empirical study**

I. Introduction

Shared dependencies concentrate npm exposure in upstream packages and maintainer accounts [1], [2]. Limited defensive budgets require distinguishing three objectives: reviewing distinct downstream repositories, reducing exposure across hypothetical account compromises, and eliminating every observed source at a repository.

October 2026 disclosures illustrate the boundaries of account controls. npm's SlugFest weakness involved mutable repository-name trust bindings; GitHub announced expiry of unvalidated bindings on October 2 [3], [4]. LiteLLM's final investigation describes an exposed master key followed by CI secret theft and misuse of multiple developer tokens [5]. Its published artifacts differed from the official source tag [6]. Railway reported potential environment-variable exposure and temporarily paused deployments [7], [8]. These contextual cases motivate authority and CI boundaries; they do not validate our graph.

Overlapping portfolios distinguish the objectives: a second account may add little coverage but still contribute a compromise-origin scenario. We compare exact portfolio unions at deduplicated workflow-repository endpoints.

We ask: **RQ1:** How do coverage and additive exposure selections differ at equal budgets? **RQ2:** How much covered infrastructure remains reachable from unselected identities? **RQ3:** How sensitive are results to representation, endpoint units, and observation gaps?

We extend maintainer-reach analysis with five deterministic methods, 200 random orders, six budgets, representation sensitivities, and independent checks. Hybrid, exclusion, threshold, and efficacy sensitivities clarify tradeoffs. A frozen acquisition pilot supports future outcome research without a forecasting claim.

II. Related Work

Maintainer prioritization. Zimmermann et al. [1] established maintainer-union reach, greedy coverage, and maintainer vetting. Zahan et al. [2] study npm weak links; Wang [9] evaluates package-node hardening in cascade simulations. We select observed usernames and remove only their account-origin scenarios, without deleting packages or blocking other origins. Coverage's submodular greedy guarantee is established [10]; exact-union ranking instead optimizes modular exposure under equal weights and costs.

Dependency semantics. Liu et al. [11] and He et al. [12] motivate resolution-aware analysis. Pillar disclosed OpenCode's legacy npm `open` command-injection path (its maintainer `sindresorhus` ranks first by exact union in our graph) on Windows/WSL on October 19; the dependency fix shipped September 28 without a CVE [13]. This motivates version-aware inventory beyond CVE matching. Shen et al. [14] report 81.26% false positives for package-level findings under finer analysis in their studied corpus; this is not an npm-wide rate. Jayasuriya et al. [15] examine version-breaking changes in Maven. Neither package nor version reach establishes executed vulnerable code.

Workflow and authority evidence. Workflow characterization [16], ARGUS [17], and unused-dependency CI waste [18] distinguish workflow association from execution. Kalu et al. [19] document technical, organizational, and human barriers to software signing. Repository accessibility [20], npm-follower [21], and release-point reconstruction [22] motivate explicit observation and temporal limits. Our cross-sectional study measures structural objectives rather than forecasting compromise.

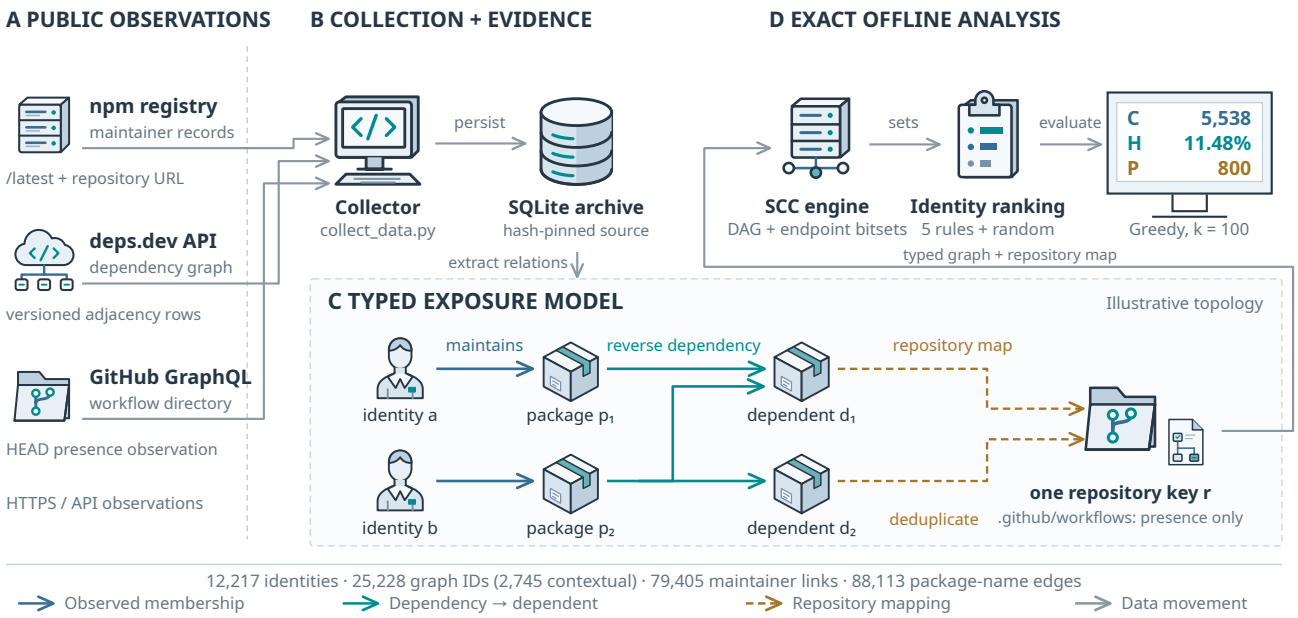


Fig. 1. Measurement architecture. Icons denote logical roles; topology is illustrative. Arrows distinguish membership, reverse dependency reach, and repository mapping. Displayed counts are empirical.

III. Data and Graph Construction

A. Collection and Observation Scope

Our pipeline seeds an npm high-impact list, expands dependencies through deps.dev, retrieves registry metadata, and checks linked GitHub repositories for a `.github/workflows` directory at HEAD (Fig. 1). The archive contains 25,228 identifiers, 22,340 version-metadata rows, 12,217 usernames, and 79,405 package–maintainer relations. Queue insertions span June 13–16, 2026; they do not establish a shared field-acquisition cutoff. We analyze the checksum-retained archive on October 20. Its 2,745 bundled-context identifiers are context-qualified graph nodes, not separate registry coordinates [23]. This expansion is not a random npm sample.

The data contain 121,885 version-edge rows and 88,113 distinct name adjacencies. We reverse all explicit declaring-to-dependency adjacencies to measure downstream reach. Stored target-node `DIRECT`/`INDIRECT` labels are relative to a resolution root; filtering by them would not recover global direct adjacencies [23].

B. Endpoints and Missing Evidence

We evaluate three endpoint universes: all package names, packages with any positive workflow observation, and distinct repository keys associated with those positive observations. The 15,233 positive packages map to 7,171 repositories. Keys are case-folded owner/name pairs with a trailing transport `.git` removed. Mapping deduplicates repositories without verifying redirects, commits, jobs, or CI installations.

There are 3,081 packages without an observed maintainer mapping and 3,772 without a completed workflow flag. A stored zero flag can conflate an absent directory with an unavailable repository. We therefore use positive observations for workflow endpoints and do not treat zero or missing values as proof of safety or workflow absence. Of the 7,171 positive repositories, 7,158 are reachable from an observed maintainer portfolio. Coverage percentages use all 7,171; all-source exclusion requires a nonempty observed source set.

C. Two Graph Representations

Our primary graph collapses retained versions into package-name nodes. For sensitivity analysis, we construct 40,806 exact (package, version) nodes and permit traversal only when adjacent version keys match. Each maintained package seeds every version present in that graph; packages without a version node retain only their eligible seed endpoint. Reach includes maintained seeds, and the same positive endpoint definition is used for both representations. Version continuity can remove paths created by joining incompatible versions, but both graphs combine retained resolution contexts and acquisition times. The version graph is a sensitivity representation, not a historical installation or ground truth.

IV. Objectives and Experimental Method

A. Maintainer Exposure Sets

Let M be the observed usernames and P_m the packages linked to m. For an endpoint universe U, we construct

$$E_m = \bigcup_{p \in P_m} \mathcal{D}_U(p), \qquad A_e = \{m \in M : e \in E_m\}, \quad (1)$$

where $\mathcal{D}_U(p)$ contains reverse-reachable endpoints of p, including its eligible seed endpoint. Repository endpoints are deduplicated across every package and path. We condense strongly connected components and propagate exact integer

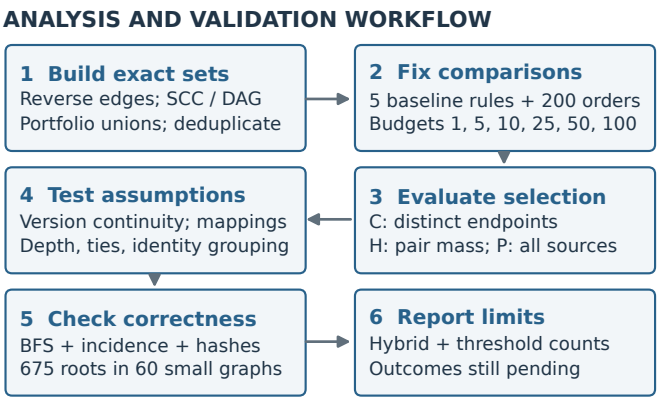


Fig. 2. Analysis methodology and validation stages. Baseline orders are reused across budgets; sensitivities are fixed-graph comparisons.

bitsets through the resulting directed acyclic graph. A separate propagation of source-identity bitsets checks the endpoint incidence relation.

B. Three Quantities for the Same Selection

For selected identities $S \subseteq M$, distinct endpoint coverage is

$$C(S) = \left| \bigcup_{m \in S} E_m \right|. \tag{2}$$

It measures the breadth of structural scenarios represented by a review or control selection. Overlap across identities contributes once.

To model source-account hardening, suppose exactly one observed username is selected uniformly as a hypothetical compromise origin. Assume a perfect control prevents that origin when selected, while all other scenarios and graph paths remain unchanged. The proportional reduction in expected endpoint count is then

$$H(S) = \frac{\sum_{m \in S} |E_m|}{\sum_{m \in M} |E_m|}. \tag{3}$$

The fixed uniform prior cancels in this ratio. It is a stipulated weighting, not an estimated account-compromise probability. Descending exact $|E_m|$ is optimal for H under equal costs because its numerator is additive. Hardening does not remove a node or protect other origins passing through its packages.

Finally, we count endpoints for which every observed source has been selected:

$$P(S) = \sum_{e \in U} \mathbf{1}[A_e \neq \varnothing \ \wedge \ A_e \subseteq S]. \tag{4}$$

We call this *all-source exclusion*. Endpoints without observed sources are excluded rather than counted as protected. Under the idealized model, $C(S) - P(S)$ covered endpoints remain reachable from an unselected source. Neither quantity establishes real control effectiveness.

C. Selection Rules and Budget Grid

We compare portfolio size, summed per-package reach, maximum per-package reach, exact portfolio-union reach, and greedy maximum marginal coverage. The sum baseline double-counts overlap within a portfolio. Greedy repeatedly adds the identity contributing the most newly covered endpoints. Coverage is monotone submodular, so this established method has the standard $1 - 1/e$ lower bound relative to optimal coverage [10]; that guarantee does not apply to H or P. The additional exclusion-oriented heuristic has no global optimality guarantee.

Selections are nested at budgets $1, 5, 10, 25, 50, 100$, with equal identity costs. Deterministic ties use ascending username order. We sample 200 seeded uniform orders without replacement, reusing each order across endpoint universes, and calculate exact random expectations from source multiplicities using hypergeometric probabilities. Random central 95% simulation ranges describe selection variability on this graph, not confidence intervals for npm or real attacks. Our saved analysis plan is exploratory and recorded before this execution; it is not an external preregistration.

D. Reproducibility and Checks

The read-only Python implementation follows Fig. 2 and records hashes, seeds, environment, and definitions. Breadth-first searches independently verify 288 roots and all 25,228 stored package transitive counts; dual incidence, union bounds, nested curves, and $P \leq C$ are checked. Tests cover cycles, version splicing, and 675 roots in 60 seeded small graphs. The grid contains 150 deterministic and 6,000 random budget rows across five endpoint/visibility definitions. No package payload is executed.

Three isolated full reruns reproduce aggregate results byte for byte. On Linux x86-64 with Python 3.14.8, mean wall time is 39.50 s (sample SD 0.98 s), with maximum peak RSS 1,038.4 MiB. These include extraction, baselines, sensitivities, and serialization; uncontrolled caches preclude online-cost inference.

E. Additional Structural Sensitivities

At budget 100, a hybrid greedily maximizes marginal gain of $J_\alpha(S) = \alpha C(S)/|U| + (1-\alpha)H(S)$ at $\alpha = 0.50$. A separate heuristic lexicographically prioritizes immediately completed source sets, fractional remaining-source contributions, marginal coverage, and username order. Candidates come from endpoints with one to ten remaining sources, falling back to all unselected identities.

To distinguish the earlier residual threshold from strict exclusion, define $R_+ = \{e : A_e \neq \varnothing\}$ and

$$T_k(S) = |\{e \in R_+ : |A_e \backslash S| < k\}|, \quad \Delta T_k = T_k(S) - T_k(\varnothing). \tag{5}$$

T_k is a residual-source count, not an enforced review quorum. The strict variant P_k^{strict} counts only endpoints with zero remaining sources or with $|A_e| \geq k$ and fewer than k remaining sources. It has zero baseline; all 13 source-free

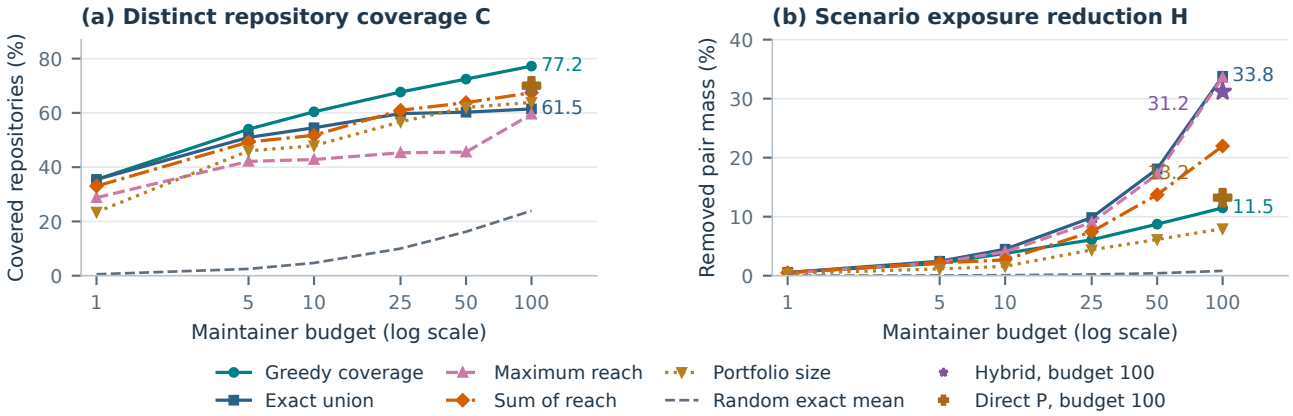


Fig. 3. Workflow-repository coverage and conditional exposure reduction favor different selections. Lines connect the six evaluated budgets. The random curve is the exact expectation; stars and plus markers show the two additional budget-100 sensitivities.

TABLE I
WORKFLOW-REPOSITORY RESULTS AT 100 IDENTITIES. C: DISTINCT COVERAGE; H: CONDITIONAL SCENARIO EXPOSURE REDUCTION; P: ALL-SOURCE EXCLUSION. RANDOM VALUES ARE 200-ORDER MEANS.

Selection	C	C (%)	H (%)	P
Portfolio	4,579	63.9	7.96	577
Summed reach	4,835	67.4	21.98	704
Maximum reach	4,268	59.5	33.59	533
Exact union	4,407	61.5	33.79	534
Greedy coverage	5,538	77.2	11.48	800
Random (mean)	1,680.6	23.4	0.80	12.8

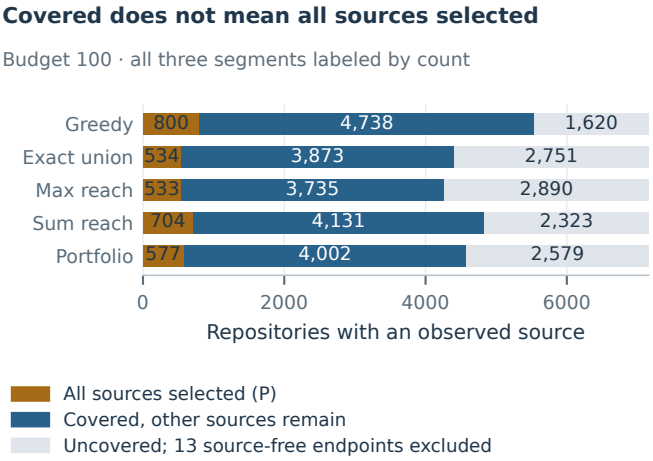


Fig. 4. Repository source-set partition at budget 100. Segment labels give all-source exclusion, covered residual exposure, and uncovered counts. Bars exclude 13 repositories without observed sources. Blue segments remain reachable through unselected identities.

endpoints are excluded. Uniform efficacy sensitivity $H_\eta = \eta H$ scales the stipulated exposure reduction without estimating a bypass rate.

V. RESULTS

A. RQ1: Coverage and Exposure Favor Different Selections

Table I reports workflow-repository outcomes at 100 identities (0.819% of usernames). Greedy covers 5,538 repositories (77.2%), versus 4,407 (61.5%) for exact union: a descriptive difference of $\Delta C = +1{,}131$ repositories (+25.7%). Exact union instead removes 33.79% of stipulated exposure, versus 11.48% for greedy ($\Delta H = -22.30$ percentage points; $\Delta P = +266$). Its H advantage follows analytically; the experiment quantifies the resulting coverage tradeoff. Maximum reach nears this H but covers fewer repositories.

Figure 3 shows the six budgets. At 100, random mean coverage is 1,680.6 (sample SD 530.7, Monte Carlo standard error $\text{MCSE}_C = 37.5$). The exact finite-population expectation is $E[C] = 1{,}716.8$; expected H is $100/12{,}217 = 0.819\%$ (SD 0.373 pp, $\text{MCSE}_H = 0.026$ pp). Differences reflect random-order variation, not sampling inference about npm.

B. RQ2: Covered Endpoints Retain Other Sources

Figure 4 partitions source-reachable repositories at budget 100. Of the 5,538 repositories covered by greedy, only 800 meet all-source exclusion; 4,738 remain reachable from an unselected identity. Across the 7,158 repositories with nonempty observed source sets, 6,358 retain such a source. Exact-union ranking excludes every observed source for 534 repositories. Neither baseline optimizes P.

Cross-identity overlap explains the difference between broad coverage and additive scenario mass. At 100 identities, $1 - C(S)/\sum_{m \in S} |E_m|$ is 89.57% for greedy and 97.18% for exact union. An observed co-maintenance pattern illustrates this masking without asserting a historical forecast: `qix` maintains only three packages (`debug`, `simple-swizzle`, `backslash`; portfolio rank #2,298) yet reaches 2,971 repositories, ranking #8 under exact union and #5 under version continuity (1,476 repositories) (including 444 repositories not reached by `sindresorhus`). Because `sindresorhus` (#1) and `tootallnate` (#8) absorb 2,527 of those end-

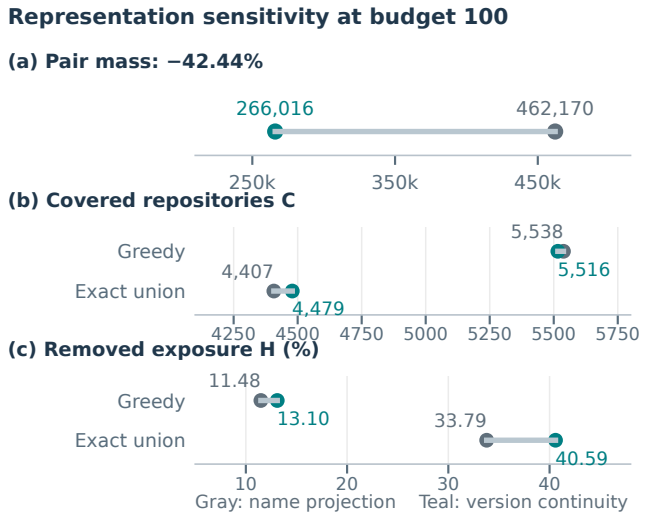


Fig. 5. Version continuity changes pair mass while aggregate coverage remains similar. The lower panels report coverage and exposure, selecting and evaluating within each representation at budget 100. Both graphs remain observed unions.

points, greedy marginal coverage skips `qix` past rank 100 while leaving all 2,971 repositories residually exposed.

C. RQ3: Representation Changes Exposure Mass

Preserving version continuity changes downstream closures for 8,587 of 25,228 package roots and removes 1,273,291 ordered root–dependent pairs introduced by package projection. Maintainer workflow-repository unions change for 2,582 usernames. Total identity–repository pairs fall from 462,170 to 266,016, a 42.44% decrease. This is representational sensitivity, not an estimate of false exploit paths in actual installations.

At 100 identities, selecting and evaluating greedy on the version graph covers 5,516 repositories, with $H = 13.10\%$ and $P = 817$. Exact-union ranking gives $C = 4,479$, $H = 40.59\%$, and $P = 527$. Evaluating the package-selected greedy identities on that version graph yields $C = 5,491$. Aggregate coverage remains similar at this budget, while additive exposure mass is substantially representation-dependent (Fig. 5).

Repository deduplication affects decisions. Selecting greedy for all-package coverage, then evaluating repository endpoints, covers 5,308 repositories; selecting for workflow-package coverage gives 5,315. Direct repository selection gives 5,538. Depth ablation gives 5,255 when selection uses portfolio endpoints alone and 5,488 with one-hop evidence, both evaluated against full reach. Endpoint units and depth therefore matter with the final evaluation denominator held fixed.

VI. Robustness and Research Implications

A. Observation and Selection Sensitivities

We exclude 60 positive observations whose stored repository key disagrees with a lexical parse of the source URL, reducing the endpoint universe by 24 repositories (7,171 to 7,147); greedy at 100 covers 5,524, with $P = 799$. Flags are not transferred without reacquisition; this check cannot certify other remote associations.

Removing all bundled-context nodes leaves every workflow-repository source set unchanged, including all 462,170 identity–repository pairs. Thus repository C, H, and P remain invariant for fixed selections. Workflow-package incidence changes by 28 pairs for one endpoint, which repository deduplication absorbs; contextual nodes should therefore not be assumed irrelevant to every endpoint definition.

Reversing tie order yields $C = 5,537$ and $P = 793$. Excluding the 1, 5, or 10 largest portfolios gives coverage of 5,522, 5,149, and 5,076. Grouping equal stored email hashes as a hypothetical control unit yields 12,147 groups with greedy coverage at 5,538. These tests reflect eligibility sensitivity rather than verified credential control.

B. Hybrid, Threshold, and Efficacy Results

At budget 100, the equal-weight hybrid gives $C = 5,020$ (70.0%), $H = 31.19\%$, and $P = 626$. Direct lexicographic exclusion selection gives $P = 1,022$ with $C = 5,020$ and $H = 13.23\%$. These sensitivities reuse the fixed graph.

For $k = 2$, $T_2(\varnothing) = 1,716$ across 7,158 source-reachable endpoints. Greedy, Exact Union, and Hybrid yield $T_2 = 2,112$, 2,137, and 2,153, respectively, giving increments $\Delta T_2 = 396$, 421, and 437. These baseline-adjusted residual counts differ from strict zero-anchored counts: $P_2^{\text{strict}} = 1,145$, 793, and 961, respectively. Neither definition measures a deployed multisigner policy.

At assumed efficacy $\eta = 0.90$, Exact Union gives $H_\eta = 30.41\%$ and Hybrid 28.07%, versus Greedy's unattenuated 11.48%. A common multiplier preserves H rankings and does not model correlated CI failures.

C. Prospective Groundwork

We sampled 100 identifiers without replacement from the 25,228-name frame (seed 20261020, frozen before requests). Metadata were captured for 90; eight bundled identifiers returned HTTP 405 and two returned HTTP 404. The cutoff is October 20, 2026, 00:48:38 Asia/Dhaka; the 90-day horizon ends January 18, 2027. All outcomes remain pending. Surveillance, reporting delays, censoring, and family-grouped splits must be fixed before outcome analysis. Unreported packages remain unlabeled; no trained forecast or performance result is reported.

VII. Validity, Limitations, and Responsible Reporting

Constructs. Membership does not establish publishing authority, account independence, token scope, or control feasibility. Reach proves neither installation nor execution or secret access. Priors, hardening efficacy, and thresholds are stipulated; signer groups and bypass privileges are unobserved.

Observation and generalization. Collection lacks a shared field cutoff, workflow commits, and extraction-root provenance. Both graphs mix resolution contexts. Missing metadata

and ambiguous zero flags can distort source sets; the high-impact-seeded sample limits npm-wide generalization.

Internal and conclusion validity. Independent traversals, dual incidence, toy cases, and generated values check correctness. Frozen data remain unchanged. Comparisons describe this snapshot; random orders measure selection variability. Reused orders and nested budgets make the 6,000 rows dependent.

Boundaries and reporting. Node.js's High-severity Brotli flaw (CVE-2026-111497) lies outside npm manifests [24]. Source-tag identity and artifact integrity establish neither equivalent nor benign builds [6]. We execute no payloads or credential probes, claim no demonstrated prevention, and retain sanitized prospective metadata without emails or package bodies.

VIII. Conclusion

At 100 identities, greedy covers 5,538 workflow repositories but excludes all observed sources for only 800; exact union yields greater conditional exposure reduction. Version continuity reduces exposure pairs by 42.44%. Hybrid and exclusion-oriented selections illustrate additional tradeoffs; threshold increments and efficacy multipliers remain hypothetical sensitivities. Version-aware inventory and release-age delays [12] complement account controls, but we establish no optimal 3-to-7 day cooldown. Railway's deployment pause motivates build-independent revocation and recovery paths [8]; their effectiveness is not measured here. Pending prospective outcomes support no forecasting claim.

Acknowledgment

We thank the npm community and the npm registry, deps.dev, and GitHub for public artifacts and interfaces. A large language model (LLM) supported drafting, code revision, analysis checks, and source verification.

References

[1] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *28th USENIX Security Symposium*. USENIX Association, 2019, pp. 995–1010. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

[2] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" arXiv:2112.10165v2, 2022, ICSE SEIP 2022; DOI: 10.1145/3510457.3513044. [Online]. Available: https://arxiv.org/abs/2112.10165v2

[3] Bloom Security, "SlugFest: Millions of AI agents install npm packages exposed to malicious takeover through GitHub," October 2026. [Online]. Available: https://bloom.security/blog/slugfest-millions-of-ai-agents-download-exposed-npm-packages

[4] GitHub, "Unvalidated npm trusted publishing configurations now expire," October 2026. [Online]. Available: https://github.blog/changelog/2026-10-02-unvalidated-npm-trusted-publishing-configurations-now-expire/

[5] O. Jensen and M. Wang, "The attack chain behind the v1.107.0 LiteLLM release," October 2026. [Online]. Available: https://docs.litellm.ai/blog/security-incident-oct-21-2026-attack-chain

[6] LiteLLM Project, "Contained: Oct 21, 2026 LiteLLM release v1.107.0 compromised," October 2026. [Online]. Available: https://docs.litellm.ai/blog/security-incident-oct-21-2026

[7] Railway, "Preliminary report: Oct 21st security incident," October 2026. [Online]. Available: https://blog.railway.com/p/preliminary-report-october-security-incident

[8] J. Cooper, "Security disclosure: Wednesday october 21st," October 2026. [Online]. Available: https://blog.railway.com/p/security-disclosure-wednesday-october-21st

[9] Z. Wang, "Defending the core: A centrality-based protection strategy for supply chain security in npm dependency network," arXiv:2606.14036, 2026, preprint. [Online]. Available: https://arxiv.org/abs/2606.14036

[10] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, pp. 265–294, 1978.

[11] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng, "Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem," in *44th International Conference on Software Engineering*, 2022. [Online]. Available: https://arxiv.org/abs/2201.03981v2

[12] H. He, B. Vasilescu, and C. Kästner, "Pinning is futile: You need more than local dependency versioning to defend against supply chain attacks," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, 2025, article FSE013. [Online]. Available: https://arxiv.org/abs/2502.06662

[13] E. Cohen, "Open and shut: Remote code execution in opencode via MCP OAuth," Pillar Security research disclosure, October 2026. [Online]. Available: https://www.pillar.security/blog/open-and-shut-remote-code-execution-in-opencode-via-mcp-oauth

[14] Y. Shen, X. Gao, H. Sun, and Y. Guo, "Understanding vulnerabilities in software supply chains," *Empirical Software Engineering*, vol. 30, no. 1, p. 20, 2025. [Online]. Available: https://gaoxiang9430.github.io/papers/emse.pdf

[15] D. Jayasuriya, S. Ou, S. Hegde, V. Terragni, J. Dietrich, and K. Blincoe, "An extended study of syntactic breaking changes in the wild," *Empirical Software Engineering*, vol. 30, no. 2, p. 42, 2025.

[16] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry, "Characterizing the security of GitHub CI workflows," in *31st USENIX Security Symposium*. USENIX Association, 2022, pp. 2747–2763. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

[17] S. Muralee, I. Koishybayev, A. Nahapetyan, G. Tystahl, B. Reaves, A. Bianchi, W. Enck, A. Kapravelos, and A. Machiry, "ARGUS: A framework for staged static taint analysis of GitHub workflows and actions," in *32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 6983–7000. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/muralee

[18] N. R. Weeraddana, M. Alfadel, and S. McIntosh, "Dependency-induced waste in continuous integration: An empirical study of unused dependencies in the npm ecosystem," in *International Symposium on the Foundations of Software Engineering (FSE)*, 2024, pp. 116:1–116:23. [Online]. Available: https://rebels.cs.uwaterloo.ca/confpaper/2024/07/16/dependency-induced-waste-in-continuous-integration.html

[19] K. G. Kalu, T. Singla, C. Okafor, S. Torres-Arias, and J. C. Davis, "An industry interview study of software signing for supply chain security," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025, pp. 81–100. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/kalu

[20] A. Tsakpinis and A. Pretschner, "Analyzing the accessibility of GitHub repositories for PyPI and NPM libraries," in *28th International Conference on Evaluation and Assessment in Software Engineering*, 2024. [Online]. Available: https://arxiv.org/abs/2404.17403v2

[21] D. Pinckney, F. Cassano, A. Guha, and J. Bell, "npm-follower: A complete dataset tracking the NPM ecosystem," arXiv:2308.12545, 2023. [Online]. Available: https://arxiv.org/abs/2308.12545

[22] I. Rahman, J. Marley, R. Paramitha, and L. Williams, "CHRONO-RESOLUTION: A dependency resolution dataset at release points for npm, PyPI, and crates.io packages," arXiv:2607.15315, 2026, ICSME 2026 Tool Demonstration and Data Showcase Track. [Online]. Available: https://arxiv.org/abs/2607.15315

[23] Open Source Insights, "deps.dev API v3: GetDependencies," Official API documentation, accessed October 20, 2026. [Online]. Available: https://docs.deps.dev/api/v3/

[24] The Node.js Project, "Wednesday, october 21, 2026 security releases," October 2026. [Online]. Available: https://nodejs.org/en/blog/vulnerability/october-2026-security-releases